

MP - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continuous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4012 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 3979 values"
## [1] "33 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	504	30	120	NA	NA	654
2	58	NA	402	91	98	649
3	290	NA	150	195	15	650
4	281	NA	214	60	36	591
5	208	NA	197	65	34	504
6	191	1	126	66	19	403
7	166	NA	92	22	16	296
8	79	NA	48	24	4	155
9	69	NA	2	NA	6	77

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7706422	0.0458716	0.1834862	NA	NA	654
2	0.0893683	NA	0.6194145	0.1402157	0.1510015	649
3	0.4461538	NA	0.2307692	0.3000000	0.0230769	650
4	0.4754653	NA	0.3620981	0.1015228	0.0609137	591
5	0.4126984	NA	0.3908730	0.1289683	0.0674603	504
6	0.4739454	0.0024814	0.3126551	0.1637717	0.0471464	403
7	0.5608108	NA	0.3108108	0.0743243	0.0540541	296
8	0.5096774	NA	0.3096774	0.1548387	0.0258065	155
9	0.8961039	NA	0.0259740	NA	0.0779221	77

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=s
```

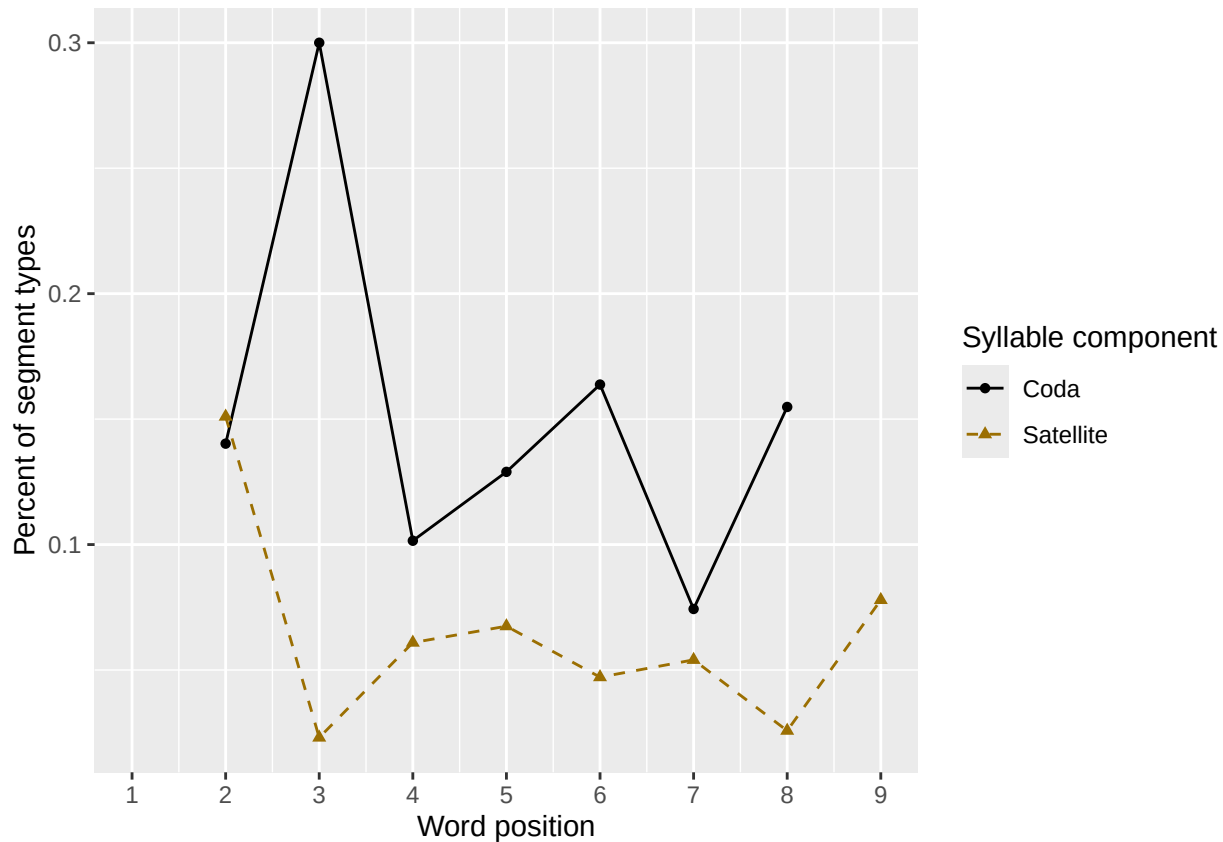
Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.945 0.964 0.927 NA      NA      NA      NA      NA      NA
## 2      5 0.966 0.989 0.978 0.955 NA      NA      NA      NA      NA
## 3      6 0.962 0.981 0.971 0.98  0.941 NA      NA      NA      NA
## 4      7 0.933 0.981 0.933 0.933 0.971 0.942 NA      NA      NA
## 5      8 0.889 0.972 0.951 0.965 0.958 0.958 0.965 NA      NA
## 6      9 0.925 0.962 0.962 0.962 0.975 0.925 0.949 0.925 NA
## 7     10 0.987 1      0.961 0.961 0.96  0.947 0.946 0.96  0.896

```

```
# len/pos table
```

```
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))

```

```
pos_len_N_table
```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    55    55    55    NA    NA    NA    NA    NA    NA
## 2      5    89    89    89    89    NA    NA    NA    NA    NA
## 3      6   104   103   104   100   102    NA    NA    NA    NA
## 4      7   105   104   104   105   104   104    NA    NA    NA
## 5      8   144   142   142   143   144   143   143    NA    NA
## 6      9    80    79    79    78    79    80    79    80    NA
## 7     10    77    77    77    76    75    76    74    75    77

```

```

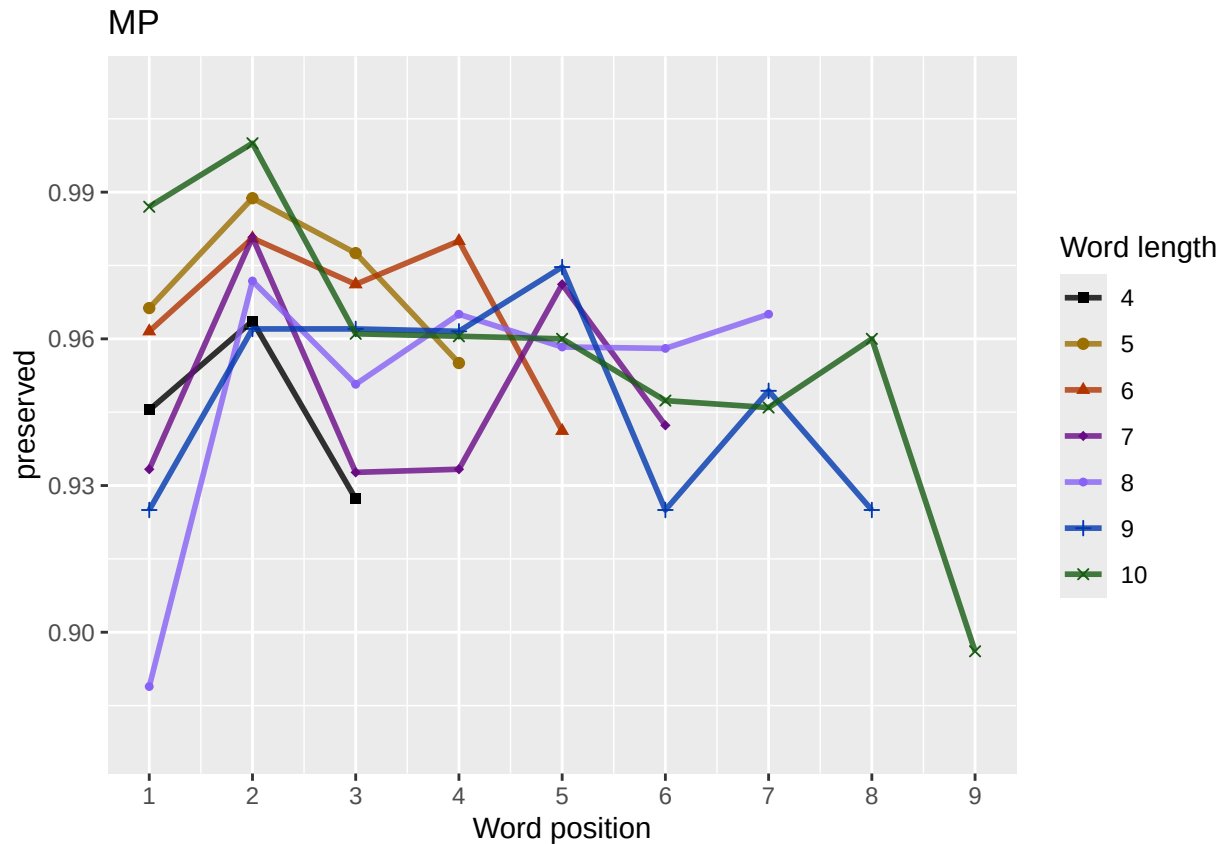
obs_linetypes <- c("solid", "solid", "solid", "solid",
                  "solid", "solid", "solid", "solid", "solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
pos_len_summary$pos <- factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved, max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)

```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 6
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.68898      -0.03697      0.28614
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1452  AIC: 1458
## log likelihood:  -726.1855
## Nagelkerke R2:  0.006338252
## % pres/err predicted correctly:  -340.1126
## % of predictable range [ (model-null)/(1-null) ]:  0.002288197
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      2.716131      -0.003774      -0.036771      0.285604
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3975 Residual
## Null Deviance:      1460
## Residual Deviance: 1452  AIC: 1460
## log likelihood:  -726.1828
## Nagelkerke R2:  0.006342704
## % pres/err predicted correctly:  -340.112
## % of predictable range [ (model-null)/(1-null) ]:  0.002290149
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      3.055
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3978 Residual
## Null Deviance:      1460
## Residual Deviance: 1460  AIC: 1462
## log likelihood:  -730.0626
## Nagelkerke R2:  3.614465e-16
## % pres/err predicted correctly:  -340.895
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)          pos

```

```

##      3.22166      -0.04251
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1459 AIC: 1463
## log likelihood: -729.34
## Nagelkerke R2: 0.001182142
## % pres/err predicted correctly: -340.7571
## % of predictable range [ (model-null)/(1-null) ]: 0.0004031923
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.38952      -0.04334
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1459 AIC: 1463
## log likelihood: -729.6096
## Nagelkerke R2: 0.0007410813
## % pres/err predicted correctly: -340.8197
## % of predictable range [ (model-null)/(1-null) ]: 0.000220248
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      stimlen:I(pos^2)      stimlen:I(pos^2)
##      3.1391540      -0.0677152      -0.0387782      0.1410479      -0.0007136      0.0007136
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3973 Residual
## Null Deviance:      1460
## Residual Deviance: 1452 AIC: 1464
## log likelihood: -725.9617
## Nagelkerke R2: 0.00670375
## % pres/err predicted correctly: -340.0745
## % of predictable range [ (model-null)/(1-null) ]: 0.002399725
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos
##      3.38738      -0.02563      -0.03436
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3976 Residual

```



```
## Null Deviance:      1460
## Residual Deviance: 1458 AIC: 1464
## log likelihood:    -729.209
## Nagelkerke R2:    0.001396462
## % pres/err predicted correctly:  -340.7362
## % of predictable range [ (model-null)/(1-null) ]:  0.0004642618
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      2.71998      0.05535      0.18168      -0.02527
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3975 Residual
## Null Deviance:      1460
## Residual Deviance: 1457 AIC: 1465
## log likelihood:    -728.6411
## Nagelkerke R2:    0.002325281
## % pres/err predicted correctly:  -340.6159
## % of predictable range [ (model-null)/(1-null) ]:  0.000816145
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                             "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ I(pos^2) + pos	1458.371	0.000000	0.000000	0.000000	0.535070	0.006332	3688980	NA	0.2861351	NA	NA
preserved ~ stimlen + I(pos^2) + pos	1460.366	1.994549	0.368888	0.4197379	0.006342	27716131	-	0.2856037	NA	-	NA
preserved ~ 1 preserved ~ pos	1462.123	3.754125	0.153039	0.0818868	0.000000	0.0055370	NA	NA	NA	NA	NA
preserved ~ stimlen	1463.219	4.848277	0.088554	0.0404738	0.000074	1389520	-	NA	NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1463.923	5.552391	1.062274	0.033320	0.500670	3.8139154	-	0.141047	0.0251939	-	-
							0.0677152		0.0387782	0.0007136	
preserved ~ stimlen + pos	1464.416	6.047010	1.048630	0.026020	0.800139	6.5387380	-	-	NA	NA	NA
							0.0256331	0.0343570			
preserved ~ stimlen * pos	1465.282	6.911165	1.031568	0.016890	0.002322	2.3719981	0.0553471	0.1816801	-	NA	NA
								0.0252747			

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ I(pos^2) + pos"
```

```
print(BestLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)           pos
##      2.68898      -0.03697       0.28614
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1452  AIC: 1458
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     4 0.950 0.957 0.961 NA      NA      NA      NA      NA      NA
## 2     5 0.950 0.957 0.961 0.962 NA      NA      NA      NA      NA
## 3     6 0.950 0.957 0.961 0.962 0.961 NA      NA      NA      NA
## 4     7 0.950 0.957 0.961 0.962 0.961 0.956 NA      NA      NA
## 5     8 0.950 0.957 0.961 0.962 0.961 0.956 0.947 NA      NA
## 6     9 0.950 0.957 0.961 0.962 0.961 0.956 0.947 0.932 NA
## 7    10 0.950 0.957 0.961 0.962 0.961 0.956 0.947 0.932 0.906
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
```

```
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
```

```

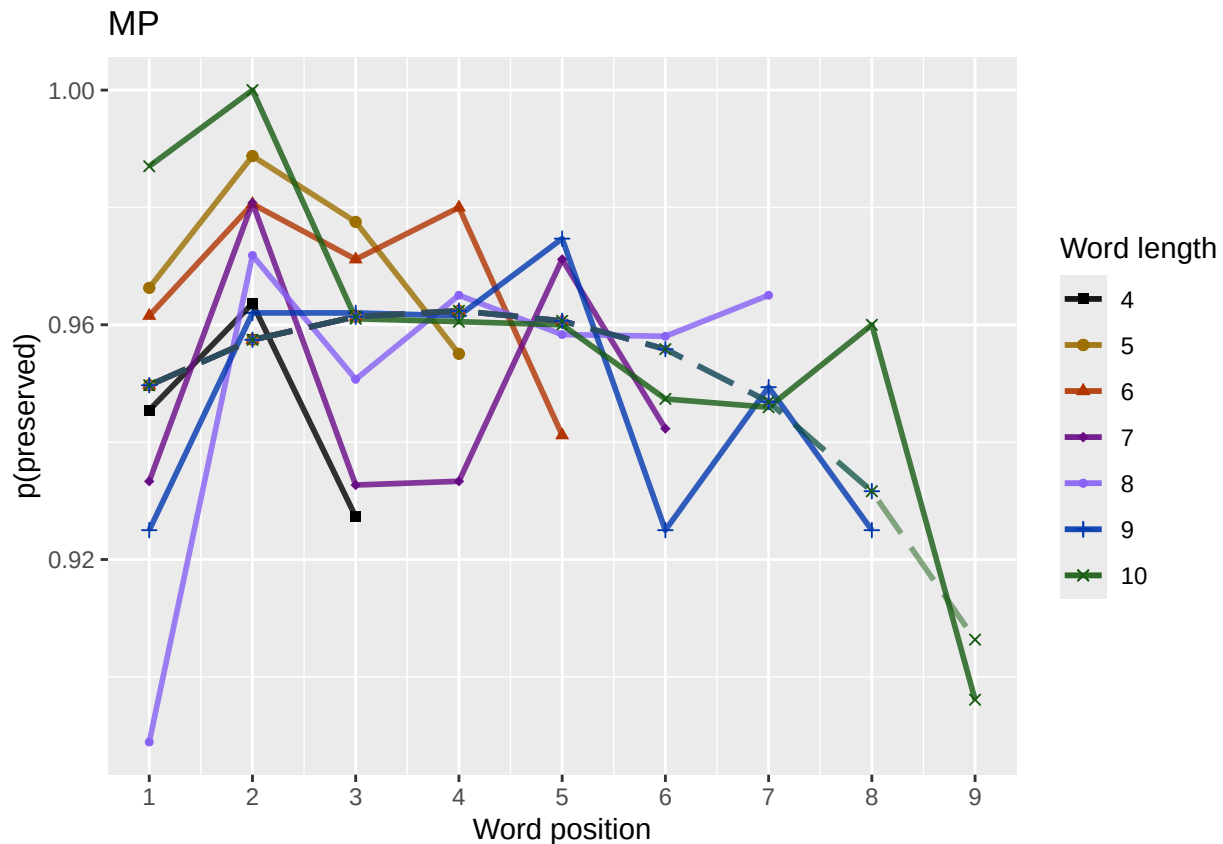
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen))) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot

```



length and position without fragments to see if this changes position² influence

```

# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models

# number responses
resp_num<-0

```

```

prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      6  654

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
  num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 6 / 654 = 0.92 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
  CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##   2.59981    -0.04394     0.36231
##
## Degrees of Freedom: 3967 Total (i.e. Null);  3965 Residual
## Null Deviance:      1391
## Residual Deviance: 1382  AIC: 1388
## log likelihood:  -691.1387
## Nagelkerke R2:   0.007624151
## % pres/err predicted correctly:  -319.9135
## % of predictable range [ (model-null)/(1-null) ]:  0.002674742
## *****

```

```

## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      2.87504      -0.03797      -0.04199      0.35699
##
## Degrees of Freedom: 3967 Total (i.e. Null); 3964 Residual
## Null Deviance:      1391
## Residual Deviance: 1382 AIC: 1390
## log likelihood: -690.8766
## Nagelkerke R2: 0.008069732
## % pres/err predicted correctly: -319.8739
## % of predictable range [ (model-null)/(1-null) ]: 0.002798037
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      3.119
##
## Degrees of Freedom: 3967 Total (i.e. Null); 3967 Residual
## Null Deviance:      1391
## Residual Deviance: 1391 AIC: 1393
## log likelihood: -695.6173
## Nagelkerke R2: 0
## % pres/err predicted correctly: -320.7742
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.62810      -0.06579
##
## Degrees of Freedom: 3967 Total (i.e. Null); 3966 Residual
## Null Deviance:      1391
## Residual Deviance: 1389 AIC: 1393
## log likelihood: -694.644
## Nagelkerke R2: 0.001658276
## % pres/err predicted correctly: -320.6206
## % of predictable range [ (model-null)/(1-null) ]: 0.0004773547
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      2.921884      -0.041070      -0.029590      0.294364      -0.001216      0.000000
##
## Degrees of Freedom: 3967 Total (i.e. Null);  3962 Residual
## Null Deviance:      1391
## Residual Deviance: 1382  AIC: 1394
## log likelihood:  -690.8651
## Nagelkerke R2:  0.008089304
## % pres/err predicted correctly:  -319.8678
## % of predictable range [ (model-null)/(1-null) ]:  0.002816984
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      pos  stimlen:pos
##      2.47334      0.07687      0.37700      -0.04461
##
## Degrees of Freedom: 3967 Total (i.e. Null);  3964 Residual
## Null Deviance:      1391
## Residual Deviance: 1386  AIC: 1394
## log likelihood:  -692.9771
## Nagelkerke R2:  0.004496635
## % pres/err predicted correctly:  -320.3052
## % of predictable range [ (model-null)/(1-null) ]:  0.001457632
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      pos
##      3.22024      -0.02619
##
## Degrees of Freedom: 3967 Total (i.e. Null);  3966 Residual
## Null Deviance:      1391
## Residual Deviance: 1391  AIC: 1395
## log likelihood:  -695.3612
## Nagelkerke R2:  0.0004363741
## % pres/err predicted correctly:  -320.7275
## % of predictable range [ (model-null)/(1-null) ]:  0.0001450662
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:

```

```
## (Intercept)      stimlen      pos
##      3.627963    -0.062226    -0.007086
##
## Degrees of Freedom: 3967 Total (i.e. Null);  3965 Residual
## Null Deviance:      1391
## Residual Deviance: 1389  AIC: 1395
## log likelihood:  -694.6277
## Nagelkerke R2:   0.001686056
## % pres/err predicted correctly:  -320.6146
## % of predictable range [ (model-null)/(1-null) ]:  0.0004958784
## *****
```

```
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                                AIC=NoFrag_LPRes$AIC,
                                row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos^2)	stimlen:I(pos^2)
preserved ~ I(pos^2) + pos	1388.277	0.000000	0.000000	0.0544079	0.9700762	22599806	NA	0.3623063	NA	0.0439415
preserved ~ stimlen + I(pos^2) + pos	1389.753	1.475888	0.4780958	0.2601222	0.0806278	75039	-	0.3569946	NA	0.0419913
preserved ~ 1	1393.235	4.957140	0.0838631	0.0456282	0.0000000	118792	NA	NA	NA	NA
preserved ~ stimlen	1393.288	5.010659	0.0816489	0.0442350	0.0016583	628099	-	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	1393.730	5.452865	0.0654524	0.0356103	0.0080823	21884	-	0.2943641	0.0056469	-
preserved ~ stimlen * pos	1393.951	6.676770	0.0585201	0.0318396	0.0044926	4733370	0.0768663	0.3770036	-	NA
preserved ~ pos	1394.722	6.445016	0.0398550	0.0216843	0.0004364	220245	NA	-	NA	NA
preserved ~ stimlen + pos	1395.255	6.978039	0.0305308	0.0166102	0.0016861	627963	-	-	NA	NA

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

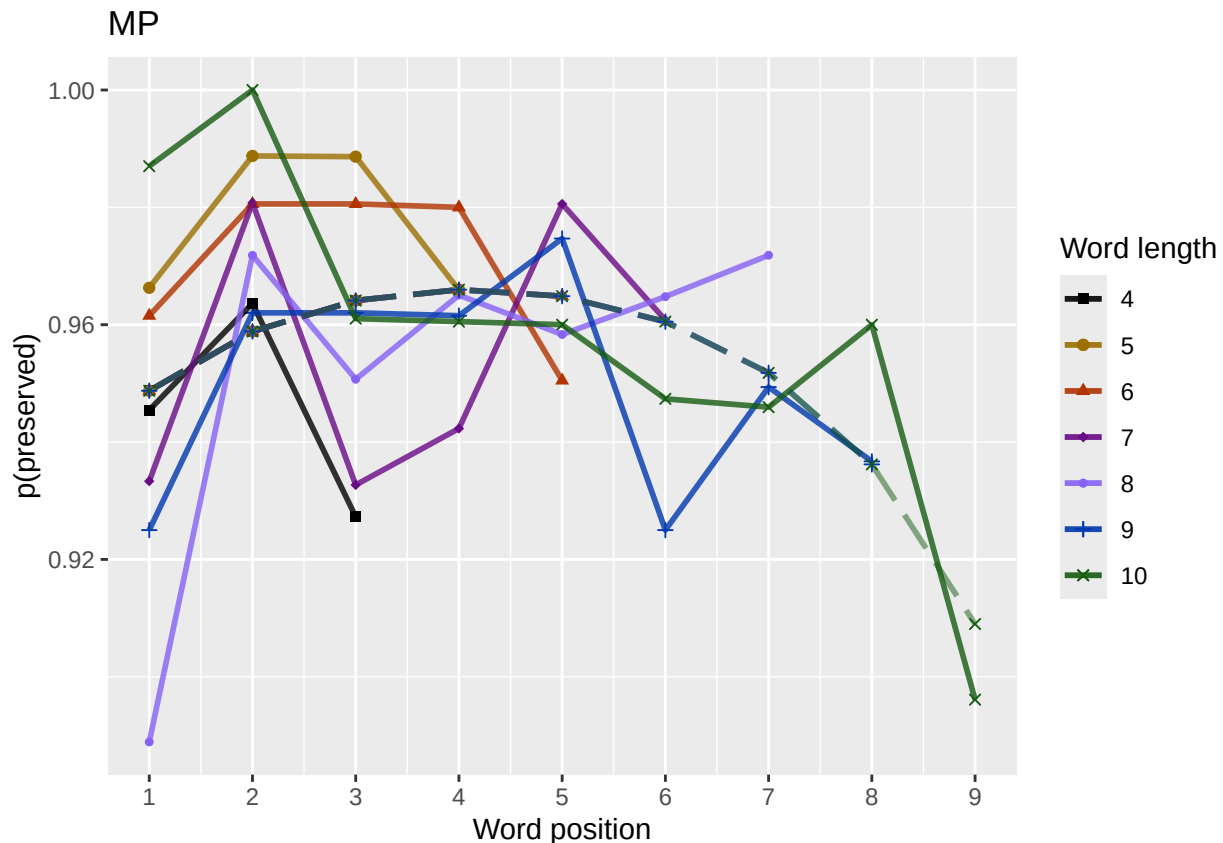
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.949 0.959 0.964 NA      NA      NA      NA      NA      NA
## 2      5 0.949 0.959 0.964 0.966 NA      NA      NA      NA      NA
## 3      6 0.949 0.959 0.964 0.966 0.965 NA      NA      NA      NA
## 4      7 0.949 0.959 0.964 0.966 0.965 0.961 NA      NA      NA
## 5      8 0.949 0.959 0.964 0.966 0.965 0.961 0.952 NA      NA
## 6      9 0.949 0.959 0.964 0.966 0.965 0.961 0.952 0.936 NA
## 7     10 0.949 0.959 0.964 0.966 0.965 0.961 0.952 0.936 0.909
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen)) +
#   geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
#   geom_point(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.88 - 1.01"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] 0
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] 0.004228518
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentage_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentage_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small sample sizes)
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
##      (Intercept)      stimlen      log_freq      I(pos^2)      pos      stimlen:log
##      2.40661      0.03330      0.43717      -0.03892      0.30338      -0
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3973 Residual
## Null Deviance:      1460
## Residual Deviance: 1440 AIC: 1452
## log likelihood: -719.9636
## Nagelkerke R2: 0.01648413
## % pres/err predicted correctly: -339.0338
## % of predictable range [ (model-null)/(1-null) ]: 0.005443561
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      I(pos^2)      pos      log_freq      I(pos^2):log_freq
##      2.751276      -0.034827      0.264591      0.297147      0.004472
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3973 Residual
## Null Deviance:      1460
## Residual Deviance: 1440 AIC: 1452
## log likelihood: -720.0794
## Nagelkerke R2: 0.01629552
## % pres/err predicted correctly: -338.9913
## % of predictable range [ (model-null)/(1-null) ]: 0.005567834
## *****
## model index: 13
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos      log_freq
##      2.41678      0.03736      -0.03768      0.29302      0.13275
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3974 Residual
## Null Deviance:      1460
## Residual Deviance: 1442 AIC: 1452
## log likelihood: -721.2359
## Nagelkerke R2: 0.01441193
## % pres/err predicted correctly: -339.279
## % of predictable range [ (model-null)/(1-null) ]: 0.004726592
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos      log_freq      I(pos
##      2.398738      0.049006      -0.037506      0.272491      0.313141

```

```

##
## Degrees of Freedom: 3978 Total (i.e. Null); 3972 Residual
## Null Deviance: 1460
## Residual Deviance: 1439 AIC: 1453
## log likelihood: -719.6647
## Nagelkerke R2: 0.01697075
## % pres/err predicted correctly: -338.8955
## % of predictable range [ (model-null)/(1-null) ]: 0.005848024
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stim
## 2.433643 0.040208 0.534383 -0.036663 0.271933
## log_freq:pos
## -0.074625
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3971 Residual
## Null Deviance: 1460
## Residual Deviance: 1438 AIC: 1454
## log likelihood: -719.0846
## Nagelkerke R2: 0.01791492
## % pres/err predicted correctly: -338.7715
## % of predictable range [ (model-null)/(1-null) ]: 0.006210787
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos log_freq pos:log_freq
## 3.25180 -0.04466 0.23611 -0.02725
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3975 Residual
## Null Deviance: 1460
## Residual Deviance: 1447 AIC: 1455
## log likelihood: -723.3788
## Nagelkerke R2: 0.01091897
## % pres/err predicted correctly: -339.7971
## % of predictable range [ (model-null)/(1-null) ]: 0.003210952
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos log_freq
## 3.20129 -0.03034 0.12804
##

```

```

## Degrees of Freedom: 3978 Total (i.e. Null); 3976 Residual
## Null Deviance: 1460
## Residual Deviance: 1449 AIC: 1455
## log likelihood: -724.4277
## Nagelkerke R2: 0.009207888
## % pres/err predicted correctly: -339.9851
## % of predictable range [ (model-null)/(1-null) ]: 0.002661071
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq stimlen:log_freq
## 3.122854 -0.008324 0.417338 -0.036707
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3975 Residual
## Null Deviance: 1460
## Residual Deviance: 1447 AIC: 1455
## log likelihood: -723.709
## Nagelkerke R2: 0.01038047
## % pres/err predicted correctly: -339.8289
## % of predictable range [ (model-null)/(1-null) ]: 0.003118098
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq
## 3.109507 -0.003408 0.130927
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3976 Residual
## Null Deviance: 1460
## Residual Deviance: 1450 AIC: 1456
## log likelihood: -724.7917
## Nagelkerke R2: 0.00861389
## % pres/err predicted correctly: -340.0382
## % of predictable range [ (model-null)/(1-null) ]: 0.002505832
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 3.10060 0.02350 -0.05211 0.24512 -0.02832
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3974 Residual
## Null Deviance: 1460
## Residual Deviance: 1447 AIC: 1457

```

```

## log likelihood: -723.2788
## Nagelkerke R2: 0.01108198
## % pres/err predicted correctly: -339.7706
## % of predictable range [ (model-null)/(1-null) ]: 0.003288596
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 3.120622 0.009441 0.417244 -0.034443 -0.036693
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3974 Residual
## Null Deviance: 1460
## Residual Deviance: 1447 AIC: 1457
## log likelihood: -723.308
## Nagelkerke R2: 0.01103448
## % pres/err predicted correctly: -339.7715
## % of predictable range [ (model-null)/(1-null) ]: 0.003285962
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq
## 3.10734 0.01436 -0.03447 0.13096
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3975 Residual
## Null Deviance: 1460
## Residual Deviance: 1449 AIC: 1457
## log likelihood: -724.3897
## Nagelkerke R2: 0.009269843
## % pres/err predicted correctly: -339.976
## % of predictable range [ (model-null)/(1-null) ]: 0.002687867
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 3.11610 0.01640 0.41592 -0.04616 -0.02593 -0
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3973 Residual
## Null Deviance: 1460
## Residual Deviance: 1446 AIC: 1458
## log likelihood: -722.825
## Nagelkerke R2: 0.01182211
## % pres/err predicted correctly: -339.6714

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.003578752
## *****
## model index: 19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##    2.68898    -0.03697    0.28614
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3976 Residual
## Null Deviance: 1460
## Residual Deviance: 1452 AIC: 1458
## log likelihood: -726.1855
## Nagelkerke R2: 0.006338252
## % pres/err predicted correctly: -340.1126
## % of predictable range [ (model-null)/(1-null) ]: 0.002288197
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##    2.716131   -0.003774   -0.036771    0.285604
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3975 Residual
## Null Deviance: 1460
## Residual Deviance: 1452 AIC: 1460
## log likelihood: -726.1828
## Nagelkerke R2: 0.006342704
## % pres/err predicted correctly: -340.112
## % of predictable range [ (model-null)/(1-null) ]: 0.002290149
## *****
## model index: 14
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##    3.055
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3978 Residual
## Null Deviance: 1460
## Residual Deviance: 1460 AIC: 1462
## log likelihood: -730.0626
## Nagelkerke R2: 3.614465e-16
## % pres/err predicted correctly: -340.895
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 16

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
## 3.22166      -0.04251
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance: 1460
## Residual Deviance: 1459 AIC: 1463
## log likelihood: -729.34
## Nagelkerke R2: 0.001182142
## % pres/err predicted correctly: -340.7571
## % of predictable range [ (model-null)/(1-null) ]: 0.0004031923
## *****
## model index: 15
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
## 3.38952      -0.04334
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance: 1460
## Residual Deviance: 1459 AIC: 1463
## log likelihood: -729.6096
## Nagelkerke R2: 0.0007410813
## % pres/err predicted correctly: -340.8197
## % of predictable range [ (model-null)/(1-null) ]: 0.000220248
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos stimlen:I(pos^2)      stimlen:I(pos^2)
## 3.1391540      -0.0677152      -0.0387782      0.1410479      -0.0007136      0.0007136
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3973 Residual
## Null Deviance: 1460
## Residual Deviance: 1452 AIC: 1464
## log likelihood: -725.9617
## Nagelkerke R2: 0.00670375
## % pres/err predicted correctly: -340.0745
## % of predictable range [ (model-null)/(1-null) ]: 0.002399725
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```

##
## Coefficients:
## (Intercept)      stimlen      pos
##      3.38738      -0.02563      -0.03436
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1458  AIC: 1464
## log likelihood:  -729.209
## Nagelkerke R2:  0.001396462
## % pres/err predicted correctly:  -340.7362
## % of predictable range [ (model-null)/(1-null) ]:  0.0004642618
## *****
## model index:  18
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos  stimlen:pos
##      2.71998      0.05535      0.18168      -0.02527
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3975 Residual
## Null Deviance:      1460
## Residual Deviance: 1457  AIC: 1465
## log likelihood:  -728.6411
## Nagelkerke R2:  0.002325281
## % pres/err predicted correctly:  -340.6159
## % of predictable range [ (model-null)/(1-null) ]:  0.000816145
## *****

BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                      by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                              RunLabel,"_",CurTask,
                              "_freq_len_pos_model_summary.csv"),row.names = FALSE)
kable(FLPAICSummary)

```

[illegible]

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_stimlen	log_freq	log_pos	log_freq(I(pos^2))	pos^2	log_freq(I(pos^2))	pos	stimlen:log_freq(I(pos^2))
preserved ~ I(pos^2) + pos	1458.671138553987080487638898A	NA	NA	0.286131	NA	NA	-	NA	NA	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	1460.8468400504710050300634776131	NA	NA	0.2856037	NA	-	NA	NA	NA	NA	NA	NA	NA
preserved ~ 1	1462.1125979816002909809005537A	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ pos	1462.187509114620098021322166A	NA	NA	-	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen	1463.21129013236300750073489520	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	1463.9239624924800628630139154	NA	NA	0.1410479	NA	-	NA	NA	0.0251939	0.0007136			
preserved ~ stimlen + pos	1464.1289080710300010283987380	NA	NA	-	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * pos	1465.23255020823800267022753980553471	NA	NA	0.1810801	NA	NA	NA	NA	NA	-	NA	0.0252747	

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * log_freq + I(pos^2) + pos"
```

```
print(BestFLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)          stimlen          log_freq          I(pos^2)          pos  stimlen:log_freq
##      2.40661          0.03330          0.43717         -0.03892          0.30338          -0.0007136
```

```
##
```

```
## Degrees of Freedom: 3978 Total (i.e. Null); 3973 Residual
```

```
## Null Deviance: 1460
```

```
## Residual Deviance: 1440 AIC: 1452
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
```

```
median_freq <- median(PosDat$log_freq)
```

```
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
```

```
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
```

```
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

```
HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

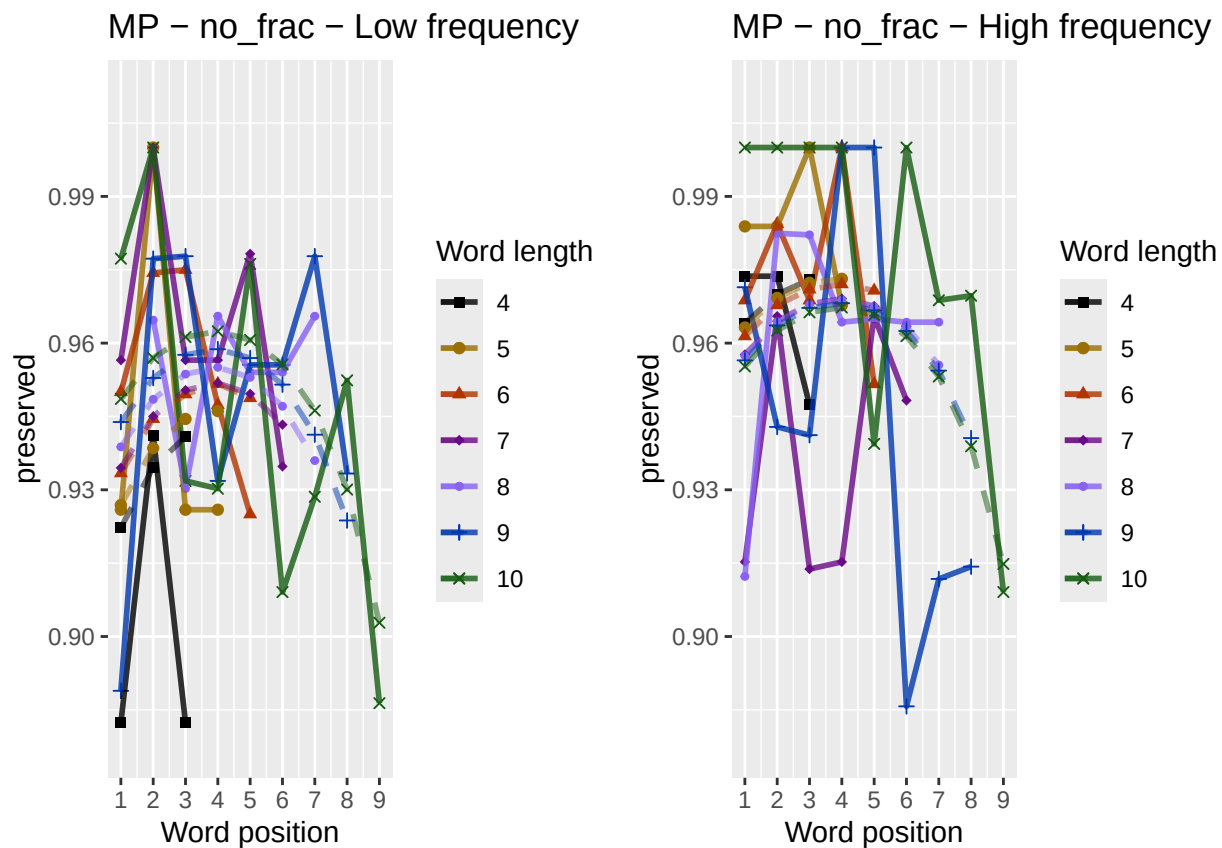
```
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFdat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)

## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_point()`).
## Warning: Removed 1 row containing missing values or values outside the scale range (`geom_line()`).

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
              CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
```

```
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##        data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.477      -1.408
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1267  AIC: 1271
## log likelihood:  -633.4867
## Nagelkerke R2:   0.1542624
## % pres/err predicted correctly:  -304.1847
## % of predictable range [ (model-null)/(1-null) ]:  0.107373
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##        data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.6365      0.1778
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1440  AIC: 1444
## log likelihood:  -719.7577
## Nagelkerke R2:   0.01681924
## % pres/err predicted correctly:  -339.1652
## % of predictable range [ (model-null)/(1-null) ]:  0.005059323
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##        data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.68898      -0.03697      0.28614
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1452  AIC: 1458
## log likelihood:  -726.1855
## Nagelkerke R2:   0.006338252
## % pres/err predicted correctly:  -340.1126
## % of predictable range [ (model-null)/(1-null) ]:  0.002288197
## *****
```

```

## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 3.055
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3978 Residual
## Null Deviance: 1460
## Residual Deviance: 1460 AIC: 1462
## log likelihood: -730.0626
## Nagelkerke R2: 3.614465e-16
## % pres/err predicted correctly: -340.895
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.22166 -0.04251
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance: 1460
## Residual Deviance: 1459 AIC: 1463
## log likelihood: -729.34
## Nagelkerke R2: 0.001182142
## % pres/err predicted correctly: -340.7571
## % of predictable range [ (model-null)/(1-null) ]: 0.0004031923
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 3.38952 -0.04334
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance: 1460
## Residual Deviance: 1459 AIC: 1463
## log likelihood: -729.6096
## Nagelkerke R2: 0.0007410813
## % pres/err predicted correctly: -340.8197
## % of predictable range [ (model-null)/(1-null) ]: 0.000220248
## *****
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

```



```

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                              CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1270.973	0.0000	1	1	0.1542623	1.476949	NA	-	NA	NA	NA
preserved ~ CumPres	1443.515	172.5421	0	0	0.0168192	0.636504	0.1778484	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1458.371	187.3976	0	0	0.0063383	0.688980	NA	NA	-	0.2861351	NA
preserved ~ 1	1462.125	191.1517	0	0	0.0000000	0.055370	NA	NA	NA	NA	NA
preserved ~ pos	1462.680	191.7066	0	0	0.0011823	0.221663	NA	NA	NA	-	NA
preserved ~ stimlen	1463.219	192.2459	0	0	0.0007413	0.389520	NA	NA	NA	NA	-
											0.0433407

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
}

```

```

BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random average"),
                                       AIC=c(mean(RndModelAIC))))
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEMModelRndDF,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
                                         RunLabel, "_", CurTask,
                                         "_syllable_component_summary.csv"), row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.9694073	523
O	0.9414951	1846
P	0.6774194	31
S	0.9692982	228
V	0.9718727	1351

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.493      -1.505
##
## Degrees of Freedom: 3719 Total (i.e. Null); 3718 Residual
## Null Deviance:      1332
## Residual Deviance: 1155 AIC: 1159

```

```

## log likelihood: -577.2665
## Nagelkerke R2: 0.1549504
## % pres/err predicted correctly: -275.6884
## % of predictable range [ (model-null)/(1-null) ]: 0.1071407
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 2.7678 0.1412
##
## Degrees of Freedom: 3719 Total (i.e. Null); 3718 Residual
## Null Deviance: 1332
## Residual Deviance: 1321 AIC: 1325
## log likelihood: -660.5673
## Nagelkerke R2: 0.009883586
## % pres/err predicted correctly: -308.0165
## % of predictable range [ (model-null)/(1-null) ]: 0.002819871
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 3.03021 -0.02463 0.14421
##
## Degrees of Freedom: 3719 Total (i.e. Null); 3717 Residual
## Null Deviance: 1332
## Residual Deviance: 1325 AIC: 1331
## log likelihood: -662.6001
## Nagelkerke R2: 0.006261641
## % pres/err predicted correctly: -308.1972
## % of predictable range [ (model-null)/(1-null) ]: 0.002236528
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) pos
## 3.40183 -0.07789
##
## Degrees of Freedom: 3719 Total (i.e. Null); 3718 Residual
## Null Deviance: 1332
## Residual Deviance: 1328 AIC: 1332
## log likelihood: -663.8704
## Nagelkerke R2: 0.003996334
## % pres/err predicted correctly: -308.483

```

```
## % of predictable range [ (model-null)/(1-null) ]: 0.001314523
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 3.089
##
## Degrees of Freedom: 3719 Total (i.e. Null); 3719 Residual
## Null Deviance: 1332
## Residual Deviance: 1332 AIC: 1334
## log likelihood: -666.1092
## Nagelkerke R2: 7.376577e-16
## % pres/err predicted correctly: -308.8903
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 3.41607 -0.04237
##
## Degrees of Freedom: 3719 Total (i.e. Null); 3718 Residual
## Null Deviance: 1332
## Residual Deviance: 1331 AIC: 1335
## log likelihood: -665.7148
## Nagelkerke R2: 0.0007043031
## % pres/err predicted correctly: -308.8254
## % of predictable range [ (model-null)/(1-null) ]: 0.0002094075
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1158.533	0.0000	1	1	0.1549503	3.492544	NA	-	NA	NA	NA
preserved ~ CumPres	1325.135	166.6016	0	0	0.0098830	2.767768	0.1411939	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1331.200	72.6672	0	0	0.0062616	1.030210	NA	NA	-	0.1442098	NA
preserved ~ pos	1331.741	73.2078	0	0	0.0039963	1.401833	NA	NA	NA	-	NA
preserved ~ 1	1334.218	75.6854	0	0	0.0000000	0.089358	NA	NA	NA	0.0778895	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ stimlen	1335.4301	76.8967	0	0	0.0007043	43.416075	NA	NA	NA	NA	- 0.0423673

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.415      -1.646
##
## Degrees of Freedom: 3196 Total (i.e. Null); 3195 Residual
## Null Deviance:      1186
## Residual Deviance: 1045 AIC: 1049
## log likelihood: -522.5675
## Nagelkerke R2: 0.139464
## % pres/err predicted correctly: -250.8823
## % of predictable range [ (model-null)/(1-null) ]: 0.09611064
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.7784      0.1351
##
## Degrees of Freedom: 3196 Total (i.e. Null); 3195 Residual
## Null Deviance:      1186
## Residual Deviance: 1179 AIC: 1183
## log likelihood: -589.7116
## Nagelkerke R2: 0.00707774
## % pres/err predicted correctly: -277.0786
## % of predictable range [ (model-null)/(1-null) ]: 0.002104213
## *****
```

```

## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
## 3.34531      -0.07645
##
## Degrees of Freedom: 3196 Total (i.e. Null); 3195 Residual
## Null Deviance: 1186
## Residual Deviance: 1182 AIC: 1186
## log likelihood: -591.1657
## Nagelkerke R2: 0.004148819
## % pres/err predicted correctly: -277.2759
## % of predictable range [ (model-null)/(1-null) ]: 0.001396247
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
## 3.02087      -0.02227      0.12306
##
## Degrees of Freedom: 3196 Total (i.e. Null); 3194 Residual
## Null Deviance: 1186
## Residual Deviance: 1180 AIC: 1186
## log likelihood: -590.1725
## Nagelkerke R2: 0.00614963
## % pres/err predicted correctly: -277.0393
## % of predictable range [ (model-null)/(1-null) ]: 0.002245236
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
## 3.04
##
## Degrees of Freedom: 3196 Total (i.e. Null); 3196 Residual
## Null Deviance: 1186
## Residual Deviance: 1186 AIC: 1188
## log likelihood: -593.2231
## Nagelkerke R2: -7.161953e-16
## % pres/err predicted correctly: -277.665
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

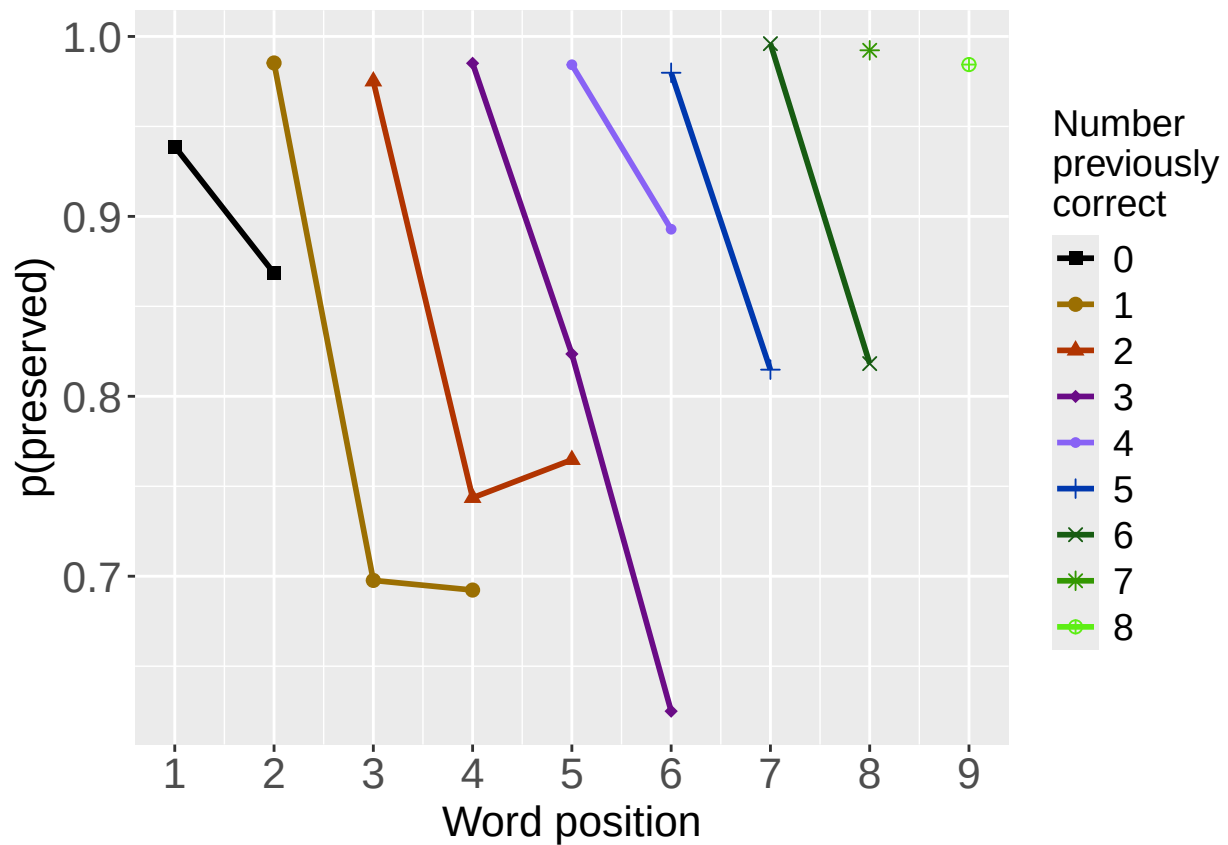
```
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      3.27859      -0.03111
##
## Degrees of Freedom: 3196 Total (i.e. Null);  3195 Residual
## Null Deviance:      1186
## Residual Deviance: 1186  AIC: 1190
## log likelihood:  -593.029
## Nagelkerke R2:  0.0003916719
## % pres/err predicted correctly:  -277.6317
## % of predictable range [ (model-null)/(1-null) ]:  0.0001193261
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~	1049.135	0.0000	1	1	0.139464	0.414682	NA	-	NA	NA	NA
CumErr								1.645883			
preserved ~	1183.423	34.2881	0	0	0.007077	2.778448	0.1351257	NA	NA	NA	NA
CumPres											
preserved ~ pos	1186.331	37.1962	0	0	0.004148	8.345312	NA	NA	NA	-	NA
										0.0764542	
preserved ~	1186.345	37.2099	0	0	0.006149	6.020869	NA	NA	-	0.1230641	NA
(I(pos^2) + pos)									0.0222677		
preserved ~ 1	1188.446	39.3111	0	0	0.000000	0.039618	NA	NA	NA	NA	NA
preserved ~	1190.058	40.9229	0	0	0.000391	3.278588	NA	NA	NA	NA	-
stimlen											0.0311107

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

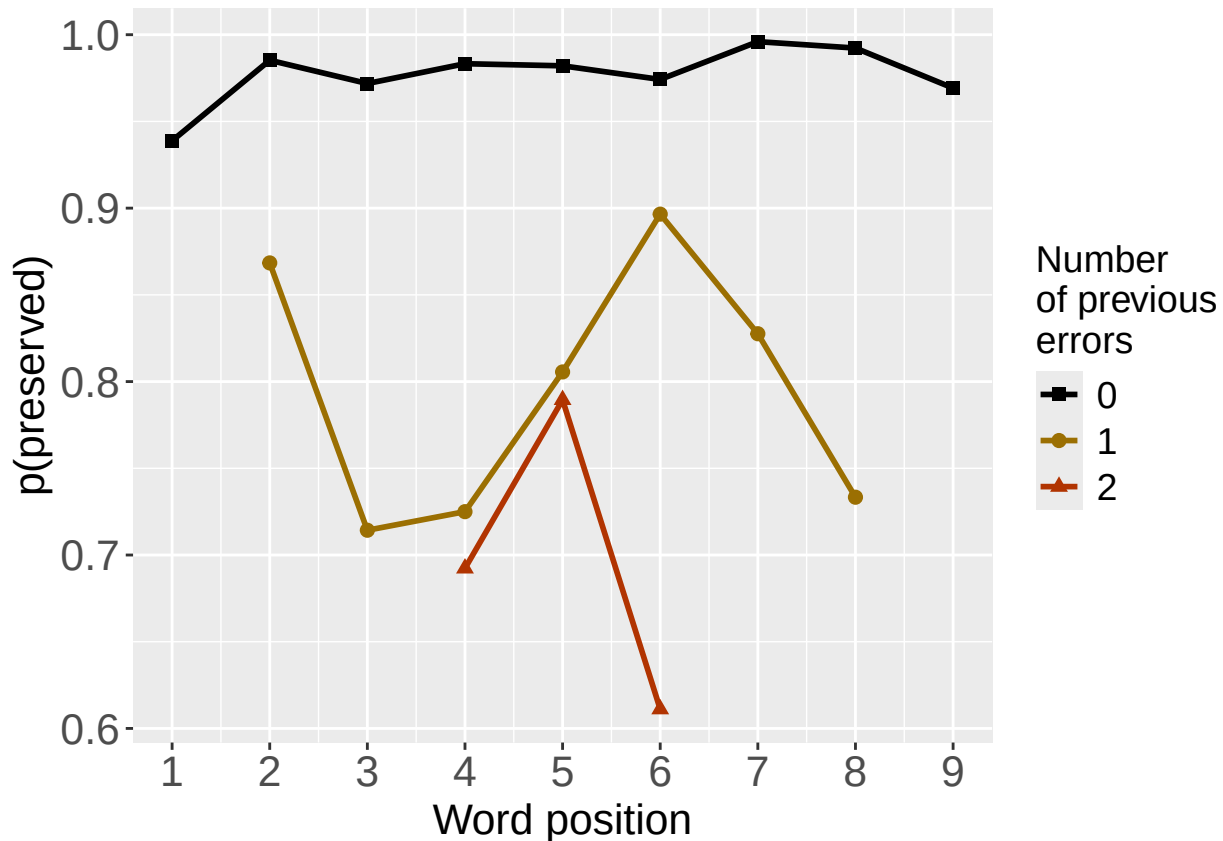
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

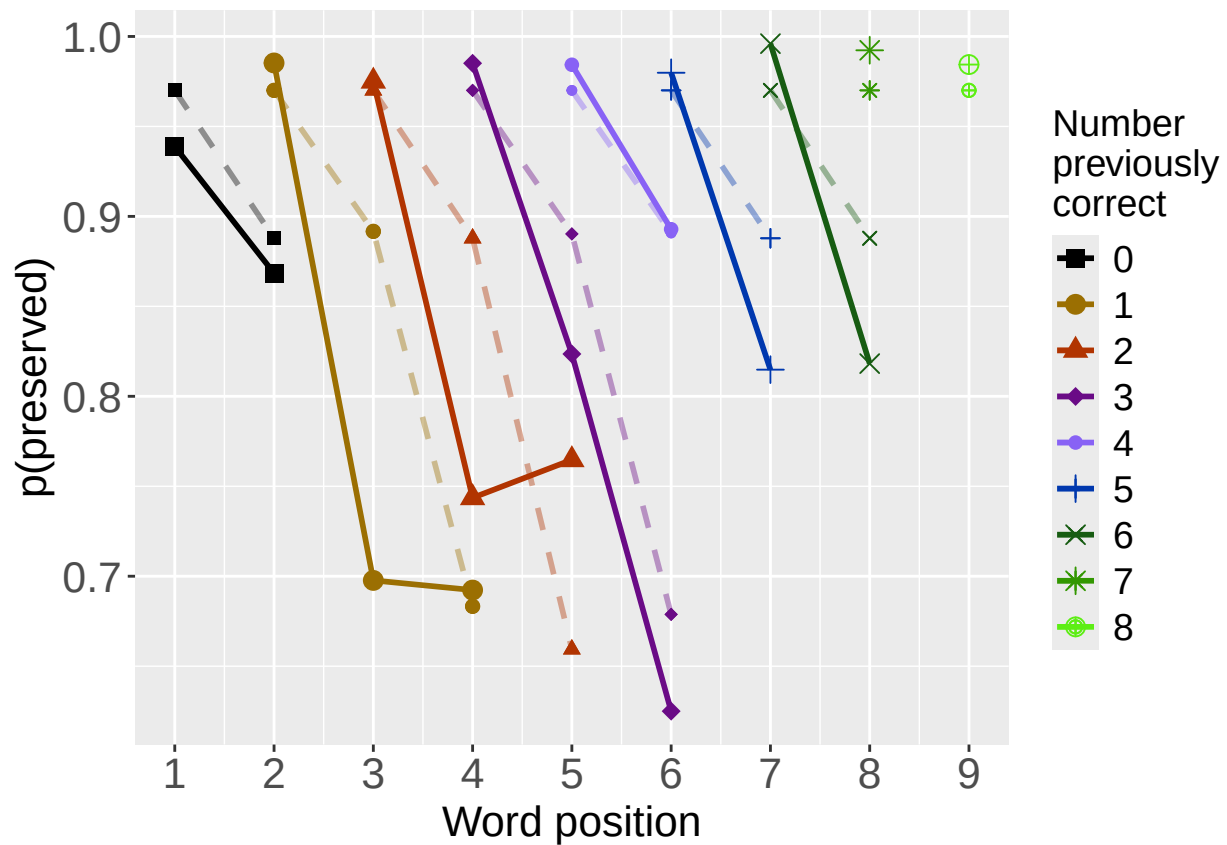
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

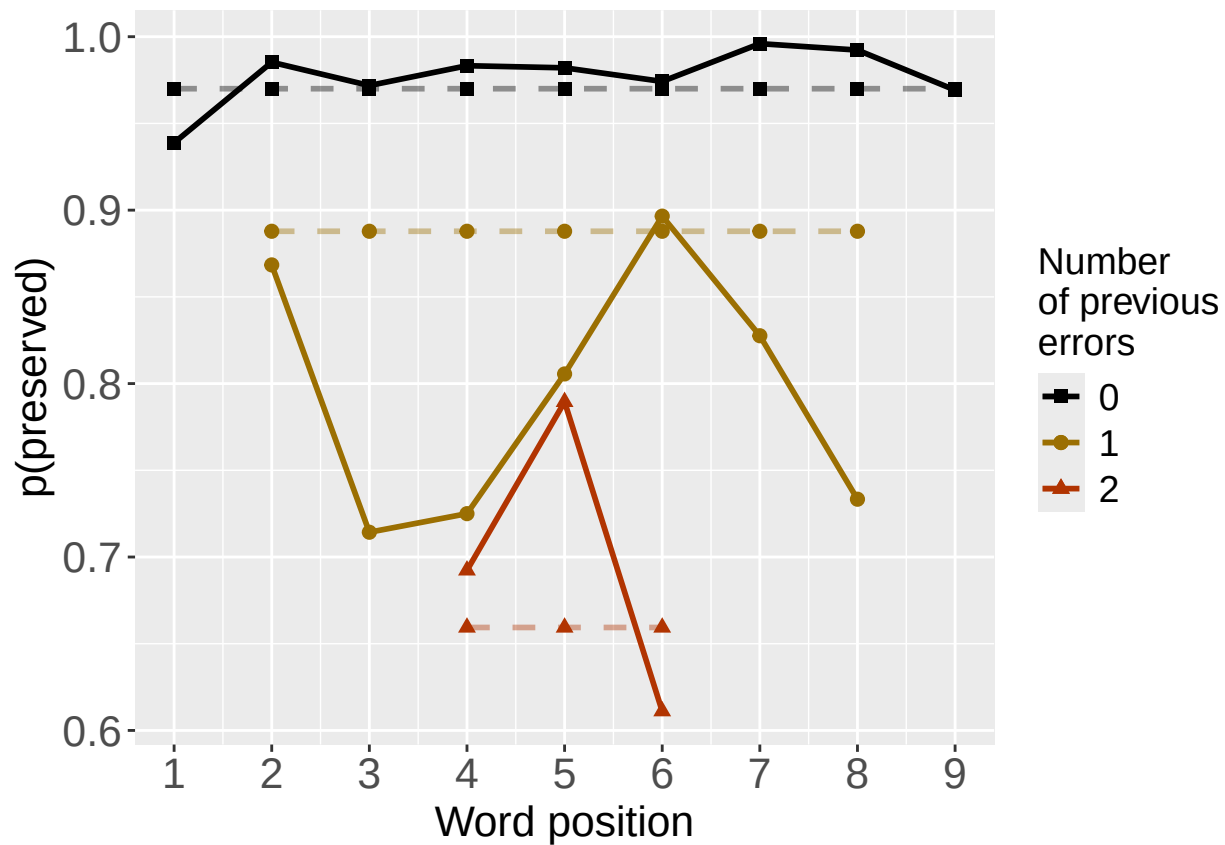
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##         data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##      2.4302      -1.5710      -0.0438      0.5134
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3975 Residual
## Null Deviance:      1460
## Residual Deviance: 1250  AIC: 1258
## log likelihood:  -624.7598
## Nagelkerke R2:  0.1678367
## % pres/err predicted correctly:  -302.8609
## % of predictable range [ (model-null)/(1-null) ]:  0.1112449
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.477      -1.408
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1267  AIC: 1271
## log likelihood:  -633.4867
## Nagelkerke R2:  0.1542624
## % pres/err predicted correctly:  -304.1847
## % of predictable range [ (model-null)/(1-null) ]:  0.107373
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.68898      -0.03697      0.28614
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1452  AIC: 1458
## log likelihood:  -726.1855
## Nagelkerke R2:  0.006338252
## % pres/err predicted correctly:  -340.1126
## % of predictable range [ (model-null)/(1-null) ]:  0.002288197
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1257.520	0.00000	1.0000000	0.9988032	0.1678367	2.430182	-1.570987	-0.0437964	0.5134103
preserved ~ CumErr	1270.973	13.45382	0.0011982	0.0011968	0.1542624	3.476949	-1.408218	NA	NA
preserved ~ I(pos^2) + pos	1458.371	200.85142	0.0000000	0.0000000	0.0063383	2.688980	NA	-0.0369678	0.2861351

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 3.477 -1.408
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance: 1460
## Residual Deviance: 1267 AIC: 1271
## log likelihood: -633.4867
## Nagelkerke R2: 0.1542624
## % pres/err predicted correctly: -304.1847
## % of predictable range [ (model-null)/(1-null) ]: 0.107373
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 3.22640 -1.41824 0.03309
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3976 Residual
## Null Deviance: 1460
## Residual Deviance: 1267 AIC: 1273
## log likelihood: -633.2725
## Nagelkerke R2: 0.1545963
## % pres/err predicted correctly: -304.1422
## % of predictable range [ (model-null)/(1-null) ]: 0.1074972
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      3.38952      -0.04334
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1459  AIC: 1463
## log likelihood:  -729.6096
## Nagelkerke R2:   0.0007410813
## % pres/err predicted correctly:  -340.8197
## % of predictable range [ (model-null)/(1-null) ]:  0.000220248
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr	1270.973	0.000000	1.0000000	0.6869307	0.1542624	3.476949	- 1.408218	NA
preserved ~ CumErr + stimlen	1272.545	1.571618	0.4557509	0.3130693	0.1545963	3.226397	- 1.418239	0.0330887
preserved ~ stimlen	1463.219	192.245871	0.0000000	0.0000000	0.0007411	3.389520	NA	- 0.0433407

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      3.0417      -1.4054      0.1864
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1249  AIC: 1255
## log likelihood:  -624.3257
## Nagelkerke R2:   0.1685104
## % pres/err predicted correctly:  -302.9225
## % of predictable range [ (model-null)/(1-null) ]:  0.1110647
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.477      -1.408
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1267  AIC: 1271
## log likelihood:  -633.4867
## Nagelkerke R2:  0.1542624
## % pres/err predicted correctly:  -304.1847
## % of predictable range [ (model-null)/(1-null) ]:  0.107373
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.6365      0.1778
##
## Degrees of Freedom: 3978 Total (i.e. Null);  3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1440  AIC: 1444
## log likelihood:  -719.7577
## Nagelkerke R2:  0.01681924
## % pres/err predicted correctly:  -339.1652
## % of predictable range [ (model-null)/(1-null) ]:  0.005059323
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1254.651	0.00000	1.0000000	0.9997145	0.1685104	3.041652	- 1.405428	0.1863753
preserved ~ CumErr	1270.973	16.32206	0.0002856	0.0002855	0.1542624	3.476949	- 1.408218	NA
preserved ~ CumPres	1443.515	188.86412	0.0000000	0.0000000	0.0168192	2.636504	NA	0.1778484

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      3.0032      -1.5545      0.1381
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1256 AIC: 1262
## log likelihood: -628.2098
## Nagelkerke R2: 0.1624775
## % pres/err predicted correctly: -303.8231
## % of predictable range [ (model-null)/(1-null) ]: 0.1084305
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.477      -1.408
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1267 AIC: 1271
## log likelihood: -633.4867
## Nagelkerke R2: 0.1542624
## % pres/err predicted correctly: -304.1847
## % of predictable range [ (model-null)/(1-null) ]: 0.107373
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.22166      -0.04251
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1459 AIC: 1463
## log likelihood: -729.34
## Nagelkerke R2: 0.001182142
## % pres/err predicted correctly: -340.7571
## % of predictable range [ (model-null)/(1-null) ]: 0.0004031923
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	1262.420	0.00000	1.0000000	0.9863045	0.1624775	3.003179	-	0.1380790
+ pos							1.554529	
preserved ~ CumErr	1270.973	8.55379	0.0138857	0.0136955	0.1542624	3.476949	-	NA
							1.408218	
preserved ~ pos	1462.680	200.26044	0.0000000	0.0000000	0.0011821	3.221663	NA	-
								0.0425117

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	1254.650	0.0000001	0.0000000	0.9997145	0.1685103	0.041652	-	NA	NA	NA	0.1863753
							1.405428				
preserved ~ CumErr + I(pos^2) + pos	1257.520	0.0000001	0.0000000	0.9988032	0.1678367	0.430182	-	-	0.5134103	NA	NA
							1.5709870		0.0437964		
preserved ~ CumErr + pos	1262.420	0.0000001	0.0000000	0.9863045	0.1624775	0.003179	-	NA	0.1380790	NA	NA
							1.554529				
preserved ~ CumErr	1270.973	3.453820	0.0011982	0.0011968	0.1542624	0.476949	-	NA	NA	NA	NA
							1.408218				
preserved ~ CumErr	1270.970	0.0000001	0.0000000	0.6869307	0.1542624	0.476949	-	NA	NA	NA	NA
							1.408218				
preserved ~ CumErr	1270.973	6.322060	0.0002856	0.0002855	0.1542624	0.476949	-	NA	NA	NA	NA
							1.408218				
preserved ~ CumErr	1270.978	5.537900	0.0138857	0.0136955	0.1542624	0.476949	-	NA	NA	NA	NA
							1.408218				
preserved ~ CumErr + stimlen	1272.545	5.5716180	0.4557509	0.3130693	0.1545963	0.226397	-	NA	NA	0.0330887	NA
							1.418239				
preserved ~ CumPres	1443.515	88.86410	0.0000000	0.0000000	0.0016819	0.2636504	NA	NA	NA	NA	0.1778484
preserved ~ I(pos^2) + pos	1458.372	0.8514090	0.0000000	0.0000000	0.0063383	0.688980	NA	-	0.2861351	NA	NA
							0.0369678				
preserved ~ pos	1462.680	0.2604400	0.0000000	0.0000000	0.0011821	0.221663	NA	NA	-	NA	NA
									0.0425117		
preserved ~ stimlen	1463.219	92.2458700	0.0000000	0.0000000	0.0007431	0.389520	NA	NA	NA	-	NA
										0.0433407	

```
# explore influence of frequency and length

if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      3.0369      -1.3791      0.1974      0.1103
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3975 Residual
## Null Deviance:      1460
## Residual Deviance: 1242 AIC: 1250
## log likelihood: -621.219
## Nagelkerke R2: 0.1733273
## % pres/err predicted correctly: -303.3215
## % of predictable range [ (model-null)/(1-null) ]: 0.1098976
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      3.16492      -1.37465      0.20193      -0.01838      0.10653
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3974 Residual
## Null Deviance:      1460
## Residual Deviance: 1242 AIC: 1252
## log likelihood: -621.163
## Nagelkerke R2: 0.173414
## % pres/err predicted correctly: -303.3
## % of predictable range [ (model-null)/(1-null) ]: 0.1099604
## *****
## model index: 1

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      3.0417      -1.4054      0.1864
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1249 AIC: 1255
## log likelihood: -624.3257
## Nagelkerke R2: 0.1685104
## % pres/err predicted correctly: -302.9225
## % of predictable range [ (model-null)/(1-null) ]: 0.1110647
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      3.3855      -1.3907      0.1998      -0.0496
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3975 Residual
## Null Deviance:      1460
## Residual Deviance: 1248 AIC: 1256
## log likelihood: -623.8865
## Nagelkerke R2: 0.1691918
## % pres/err predicted correctly: -302.885
## % of predictable range [ (model-null)/(1-null) ]: 0.1111744
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      3.055
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3978 Residual
## Null Deviance:      1460
## Residual Deviance: 1460 AIC: 1462
## log likelihood: -730.0626
## Nagelkerke R2: 3.614465e-16
## % pres/err predicted correctly: -340.895
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****

```

```

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres + log_freq	1250.438	0.0000001	0.000000	0.632865	0.173327	0.036913	-	0.197441	0.1102867	NA
preserved ~ CumErr + CumPres + stimlen + log_freq	1252.326	1.888089	0.389051	0.124621	0.0917341	0.164917	-	0.201928	0.1065343	-
preserved ~ CumErr + CumPres	1254.654	4.213382	0.121639	0.076981	0.168510	0.1041652	-	0.1863753	NA	NA
preserved ~ CumErr + CumPres + stimlen	1255.773	5.335022	0.069424	0.043936	0.116919	0.1385513	-	0.1998379	NA	-
preserved ~ 1	1462.123	11.68716	0.000000	0.000000	0.000000	0.0055370	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + log_freq
##      Df Deviance   AIC
## CumErr    1   1426.3 1432.3
## CumPres    1   1262.9 1268.9
## log_freq    1   1248.7 1254.7
## <none>      1242.4 1250.4

```

```

#####
# Single deletions from best model

```

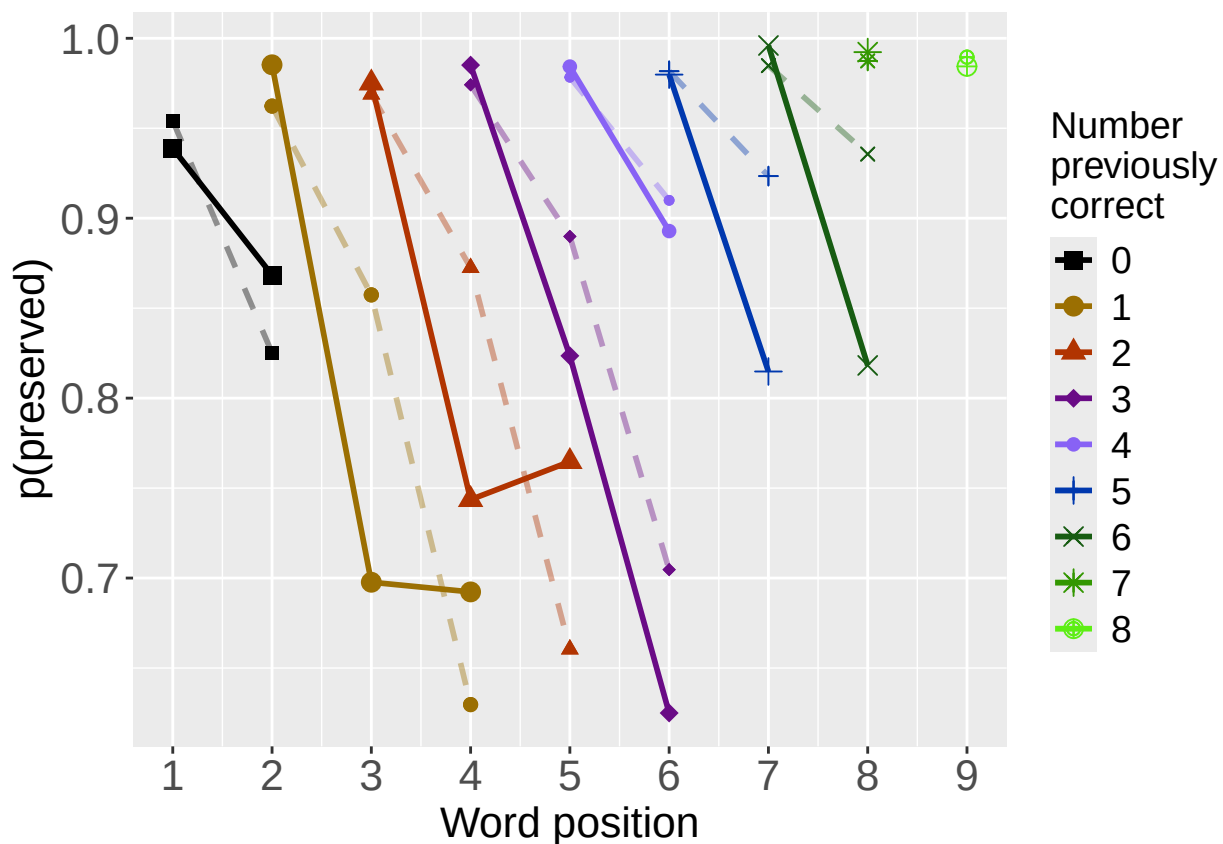
```
#####

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),
          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

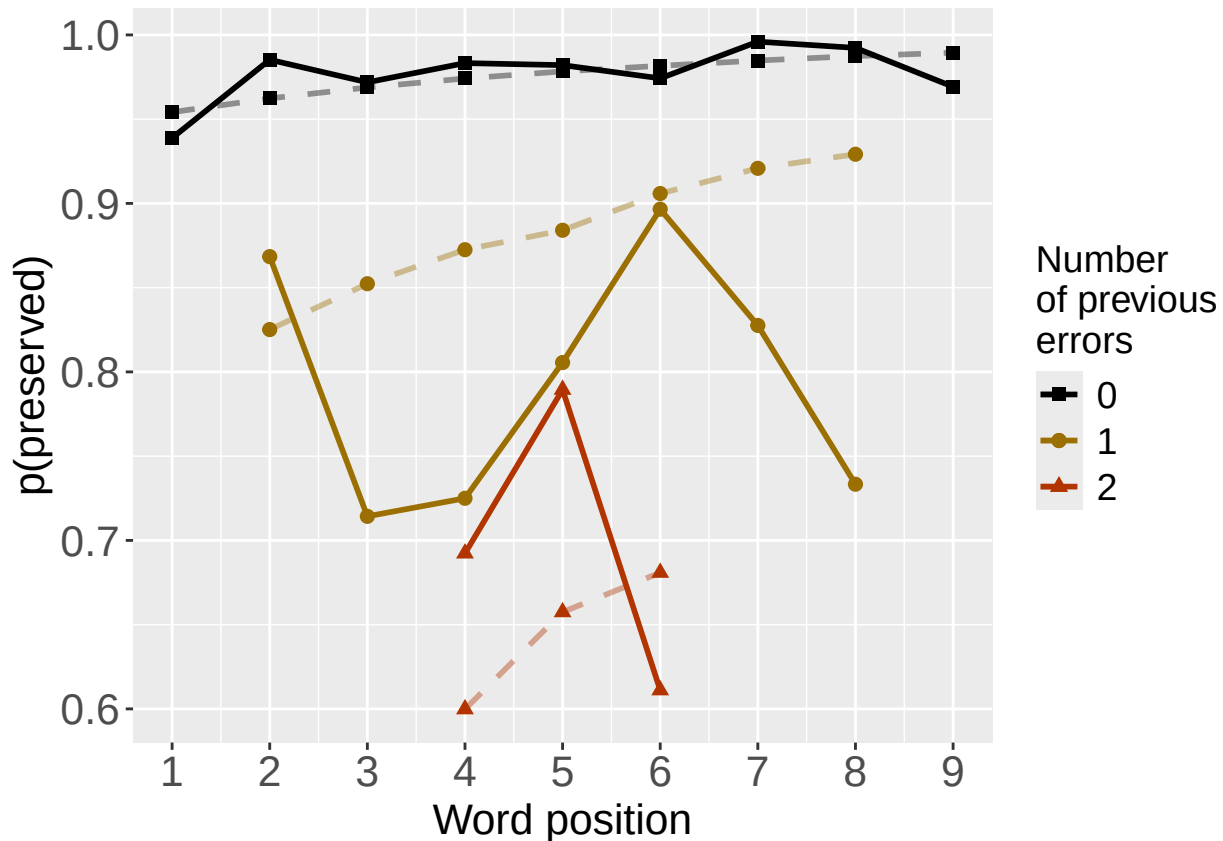
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          3.477      -1.408
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3977 Residual
## Null Deviance:      1460
## Residual Deviance: 1267 AIC: 1271
## log likelihood: -633.4867
## Nagelkerke R2: 0.1542624
## % pres/err predicted correctly: -304.1847
## % of predictable range [ (model-null)/(1-null) ]: 0.107373

```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      3.0417      -1.4054      0.1864
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3976 Residual
## Null Deviance:      1460
## Residual Deviance: 1249 AIC: 1255
## log likelihood: -624.3257
## Nagelkerke R2: 0.1685104
## % pres/err predicted correctly: -302.9225
## % of predictable range [ (model-null)/(1-null) ]: 0.1110647
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      3.0369      -1.3791      0.1974      0.1103
##
## Degrees of Freedom: 3978 Total (i.e. Null); 3975 Residual
## Null Deviance:      1460
## Residual Deviance: 1242 AIC: 1250
## log likelihood: -621.219
## Nagelkerke R2: 0.1733273
## % pres/err predicted correctly: -303.3215
## % of predictable range [ (model-null)/(1-null) ]: 0.1098976
## *****

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

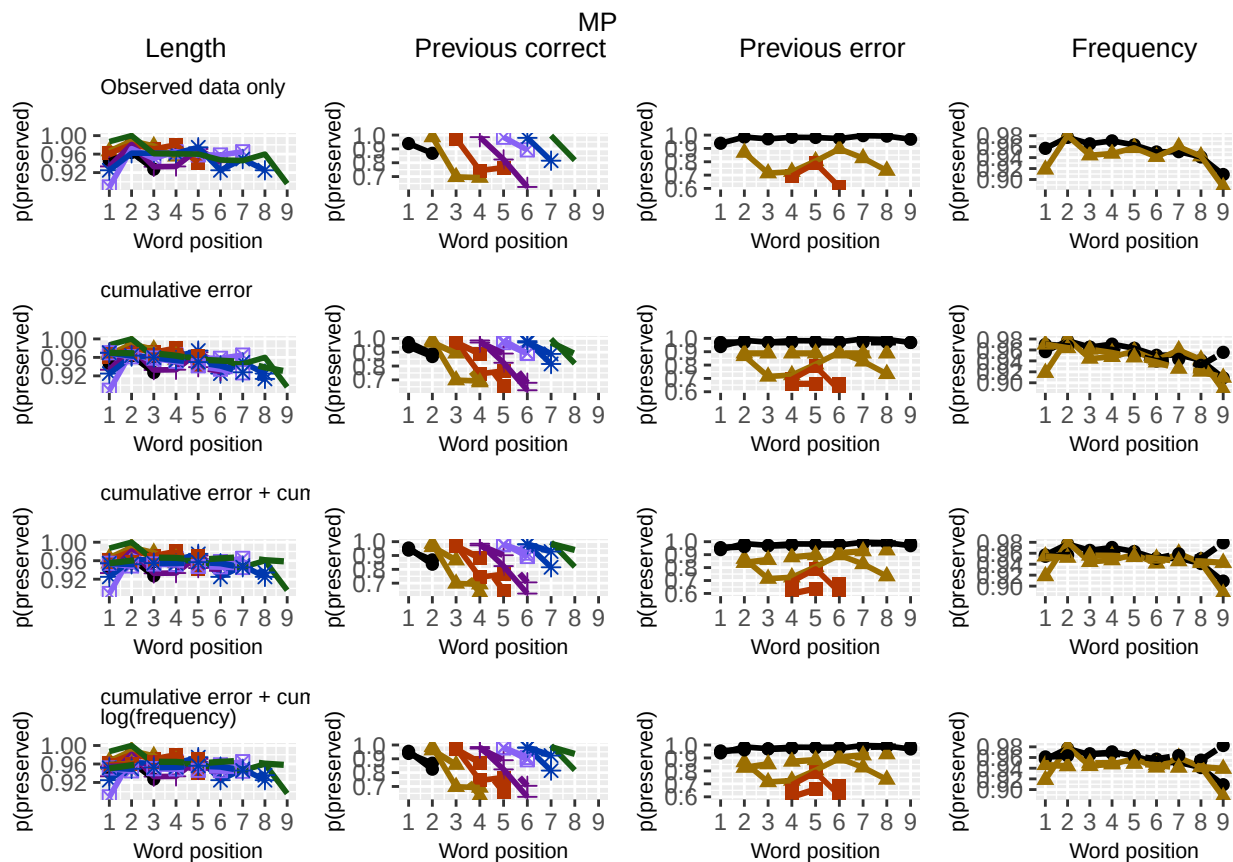
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

```

```

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`).
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot

```



```

DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                     CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)

```

	CumErr	CumPres	log_freq
McFadden	0.1291719	0.0141199	0.0057962
SquaredCorrelation	0.0461064	0.0050465	0.0020865
Nagelkerke	0.1501050	0.0164294	0.0067929
Estrella	0.0498671	0.0054492	0.0022070

	deviance	deviance_explained
CumErr + CumPres + log_freq	1242.438	217.6872
CumErr + CumPres	1248.651	211.4738
CumErr	1266.973	193.1517
null	1460.125	0.0000

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres + log_freq	14.90880	100.00000	2.854271
CumErr + CumPres	14.48326	97.14573	8.416695
CumErr	13.22844	88.72903	88.729034
null	0.00000	NA	0.000000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                model deviance deviance_explained percent_explained percent_of_explained_deviance
## CumErr + CumPres + log_freq CumErr + CumPres + log_freq 1242.438          217.6872          14.90880          100.00000
## CumErr + CumPres              CumErr + CumPres 1248.651          211.4738          14.48326          97.14573
## CumErr                        CumErr 1266.973          193.1517          13.22844          88.72903
## null                          null 1460.125           0.0000          0.00000          NA
##                                increment_in_explained
## CumErr + CumPres + log_freq          2.854271
## CumErr + CumPres                    8.416695
## CumErr                            88.729034
## null                             0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
```

```

NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.86602097
## CumPres   0.09478808
## log_freq  0.03919095

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

model	p_accounted_for	model_deviance
preserved ~ CumErr+CumPres	0.8141360	1248.651
preserved ~ CumErr+CumPres+log_freq	0.8304499	1242.438
preserved ~ CumErr	0.8324174	1266.973

model	diff_CumErr+CumPres	diff_CumErr+CumPres+log_freq	diff_CumErr
preserved ~ CumErr+CumPres	0.0000000	-0.0163139	-0.0182814
preserved ~ CumErr+CumPres+log_freq	0.0163139	0.0000000	-0.0019675
preserved ~ CumErr	0.0182814	0.0019675	0.0000000

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)

sse_table
```

```
##              model p_accounted_for model_deviance diff_CumErr+CumPres diff_CumErr+CumPres+log_freq diff_CumErr
## 1      preserved ~ CumErr+CumPres      0.8141360      1248.651           0.00000000      -0.016313904 -0.018281366
## 2 preserved ~ CumErr+CumPres+log_freq      0.8304499      1242.438           0.01631390           0.000000000 -0.001967462
## 3           preserved ~ CumErr      0.8324174      1266.973           0.01828137           0.001967462  0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```